DOCUMENTATION OF GITHUB GLINTT-NEXT TEMPLATE PACKAGE

First version of the template Package

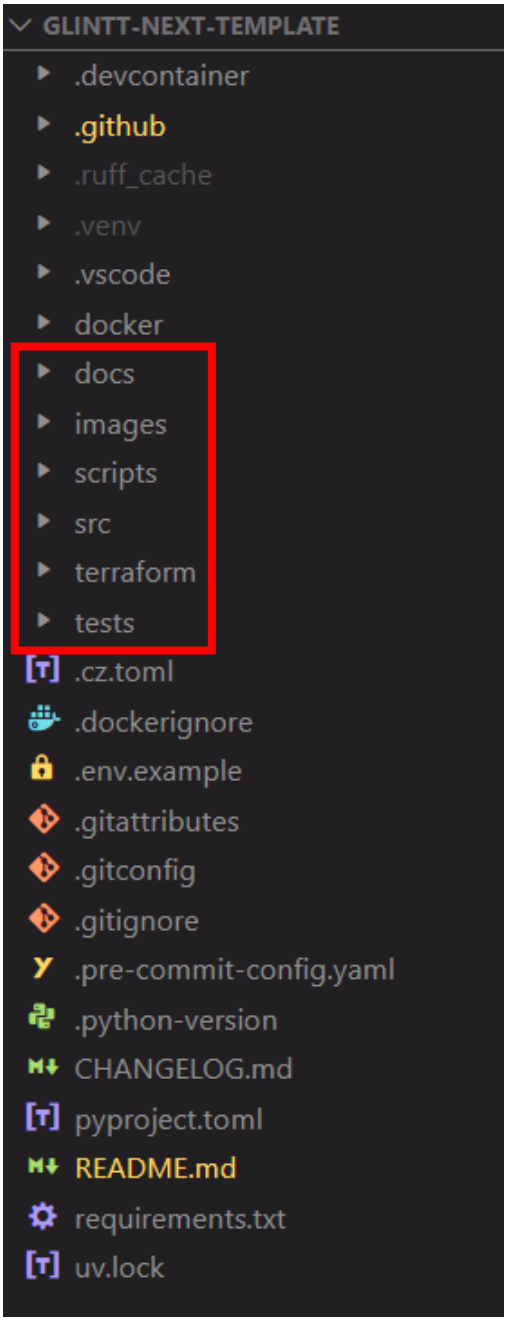
Table of Contents

- Intro
- Python Project Structure
- Package Management with UV
- Docker Containerization and Orchestration
- Pre-commit
- Vscode Config
- Commitizen
- Git Files and .github folder
- Github Specific Things
- Integration of Github with Jira
- Workflow, How to send local changes to remote repo
- Resources

Soham Malhotra

This guide aims to provide a clear overview of the Github Template Package, with its features. This documentation pertains to the very first version, with the hope that future generations can use it as a stepping-stone for future development. The guide will consist of tools explanation, workflow changes and day-to-day impacts through them. By reading this guide you should be able to jump into any template package and start developing. This first version will focus on Python specific projects, but understanding the process with which it was made can help future developers find out what they will need and how to set up their environments regardless of the programming language.

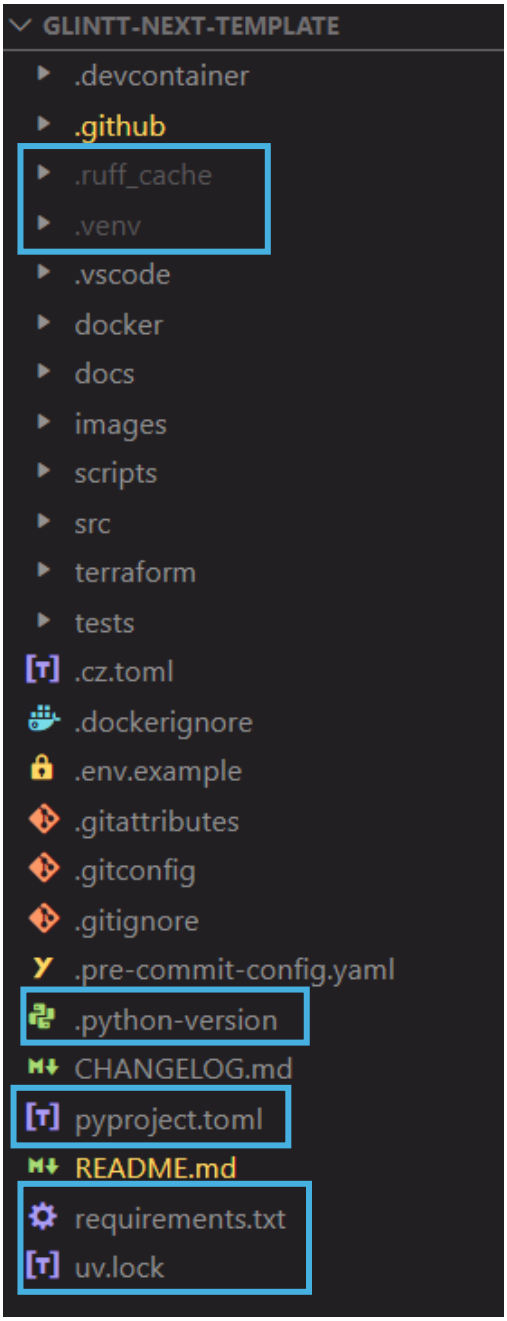
Python Project Structure



Here we see the template package opened up in VSCode, as I believe it provides a nice colour / icon coded display which will facilitate explanation from my side and comprehension from yours. The **folders marked by the colour red** are very common folders in any python package. **Docs** contain relevant documentation (for example this guide) and other useful information which isn't succinct enough to be in the README.md and focuses on more specific topics than the overview the README.md is meant to provide. **Images** is another folder where you can reference images used in the readme, images used for the frontend etc. **Scripts** generally store shell scripts, that allow you to enact specific functionality but not directly part of the project (a script that runs a cronjob to upload logs that were produced to a remote server). **Src** contains the main core of your project and all the business logic, typically it will have your main.py for example. **Terraform** is a language that allows you to write Infrastructure as Code, and is useful when pushing projects to production, and this is where those files can be saved, not as conventional of a folder but in larger industry projects it becomes a good thing to have. **Tests** is another classic package, wherein unittests, pytests and others live.

Package Management with uv

Uv is a python package manager, it controls all the dependencies and versioning of the project tools requirements. It is very fast and is a great tool as it abstracts a lot of hard tasks with controlling large dependencies on big projects. Uv is created by Astral, the same company which makes Ruff, the industry standard for Python linting and formatting.



The files generated by **uv are coloured in blue,** these are responsible for all the functionality mentioned above. The simplest file, **python-version**, just tells uv which python version to use, which will be useful when fetching relevant versions of the required packages. Requirements.txt is a class python file, not needed in the context of uv, but sometimes might be useful when you need to export dependencies to an environment that doesn't support uv. This file can automatically be created with uv, using the following command.

uv export --format requirements-txt

Uv also provides pip integration, meaning it can be use pip with the command

uv pip install <…>

However, this isn't recommended, as it partially defeats the purpose of having uv in the first place. Note that when using uv, it automatically generates the virtual environment for you, inside the. venv folder (note that if you use Windows then the files here will be correct, if you use Linux you will need to destroy this and re-run the uv start up process. Any new uv project can be started with

uv init

Then the most common commands you will use, say in the case where you will clone this project and want to set it up, or to make sure everything is correctly versioned and no conflicts

uv sync

And then to add or remove dependencies you will use

uv add <package_name>

uv remove <package_name>

You can choose to activate the virtual environment the normal way or simply run the desired command in uv's native format i.e. if you want to run

python main.py

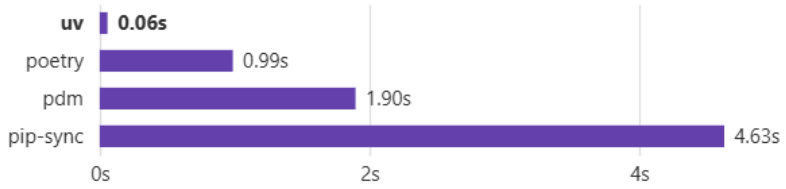
you can just say the following to run in the venv

uv run python main.py

For further information on uv, and its tools consult its documentation here.

UV

An extremely fast Python package and project manager, written in Rust.



Installing Trio's dependencies with a warm cache.

Highlights

- 🚀 A single tool to replace `pip`, `pip-tools`, `pipx`, `poetry`, `pyenv`, `twine`, `virtualenv`, and more.
- ⚡ 10-100x faster than `pip`.
- 🗂️ Provides comprehensive project management, with a universal lockfile.
- ✨ Runs scripts, with support for inline dependency metadata.
- 🌀 Installs and manages Python versions.
- 🛠️ Runs and installs tools published as Python packages.

The above picture is a screenshot from uv's documentation homepage, showing why it is better than other python package managers out there.

The **ruff_cache** is generated by ruff automatically when the linter/formatter is called, and is used to store a cache of files, say for example storing the files which have already been linted and didn't change since last check, these will be ignored, which further increases speed. Note that if that is not the desired functionality you can specify ruff to lint the whole package as well, it is all very customizable. The main commands you will be using are

uv run ruff check --fix

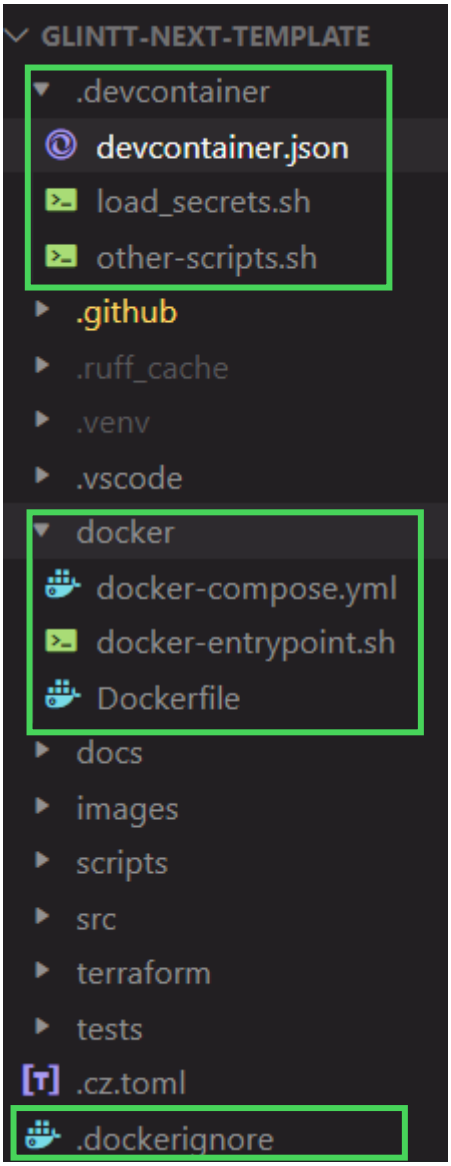
uv run ruff format

Where the first one does the linting and the second the formatting. For further information consult Ruff documentation here.

The **pyproject.tom**l is the main powerhouse of uv, and it is where all the dependencies, metadata and other information is stored. Here you can also differentiate between development and production dependencies for example, so when exporting dependencies only production-essential packages are sent. **Uv.lock** is another important

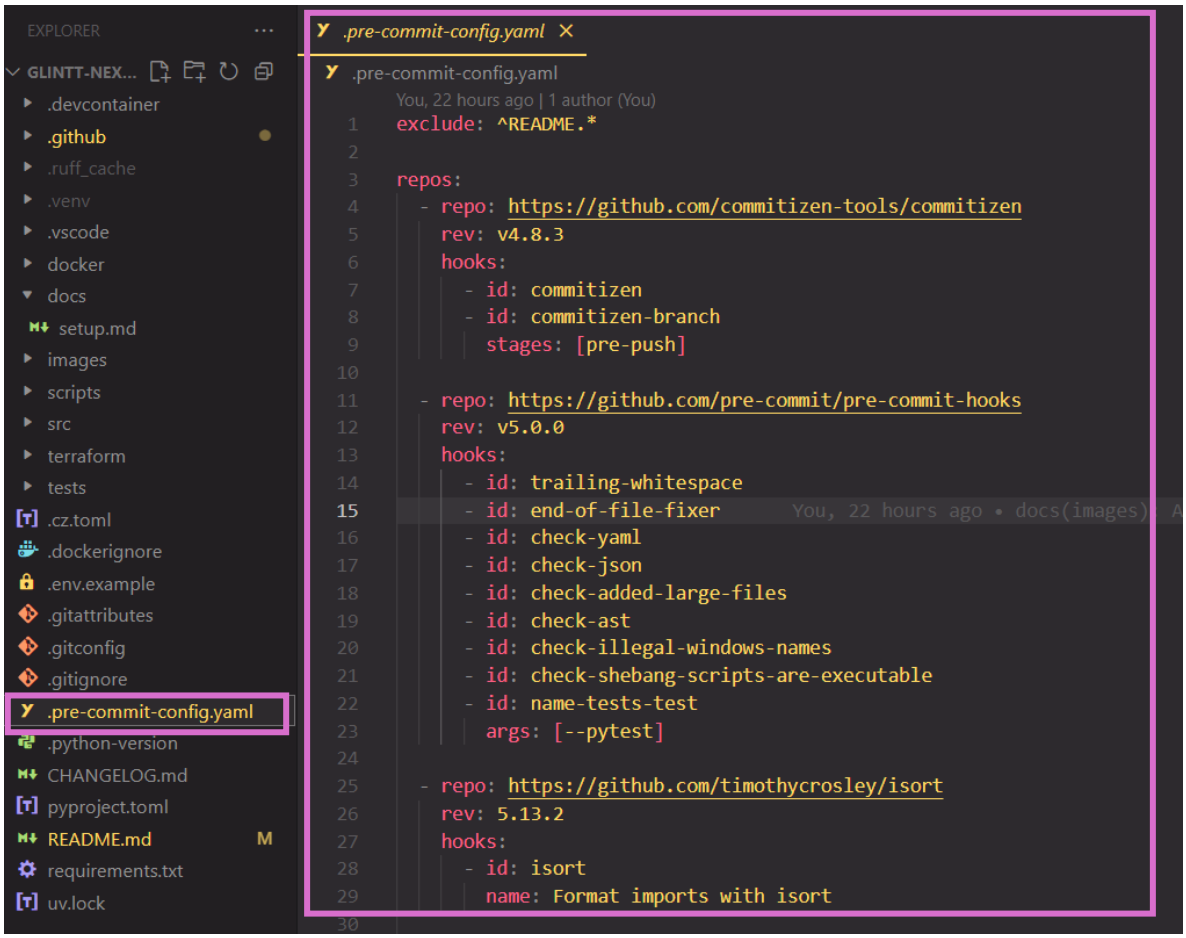
file, which keeps track of the current versions of the project, and "locks" dependencies, with a Hash of the package signature, so that it can easily reference a specific version of a specific dependency in the project. Once again, more information on all of these can be found in the uv documentation linked above or in the resources section below.

Docker Containerization + Orchestration



Files in **green are used for containerization and orchestration** of the project using Docker. Firstly, starting with the **docker** folder, this contains the files required to create a Docker image, which is essentially a "snapshot" of the required system (Linux, dependencies etc) with packages required like python and uv, so that anyone with Docker can load this project regardless of the underlying system. This image is created with the **Dockerfile**, docker-compose.yml is used to orchestrate these images, through something called containers, which are live, currently in use, instances of the image. The **docker-compose** can take multiple images, and launch them to create a whole app, say our local Project image and a default SQL database image, so that anyone can host the database and test locally / deploy from any device. **Entrypoint.sh** is just the file that is run when the container is first started. **.dockerignore** are files we don't want to have in the Docker Image, for example the ruff_cache, as firstly this can be quite a big folder and as it is a linter it isn't relevant on the production level and would only occupy space that isn't needed. Finally we have the **.devcontainer** folder, this takes containerization to another level, with its goal being that not only anyone can deploy with a uniform environment but why not test locally, develop locally as if you are in that environment. It helps set up the IDE and terminals using your defined Dockerfile to create that environment so even on development phase you are in the container. This setup is done in the **devcontainer.json** file, and the **.sh** files are useful scripts used by the devcontainer.json to set the dev container up. An interesting mention is the load_secrets.sh file, which communicates with an Azure KeyVault to get secrets from so nothing is stored on the project level.

Pre-commit



Next, we will talk about the **pre-commit package** (added as a dependency in uv) and its file which **is in the purple box.** The purpose of this file is to define some checks to run before each commit (hence the pre-commit), so every time a git commit is issued, these checks are run. If the checks fail, then the commit will not go through. This ensures that each commit has some level of code safety and correctness, and even though most people have checks on a Pull Request level, having them on a commit level makes it much easier as the amount of code to check is smaller, hence easier to fix if needed. The syntax is displayed on the right, and as you can see it is very simple git-repos which are called and used. These pre-commit hooks as they are called will need to be installed.

uv run pre-commit install --install-hooks

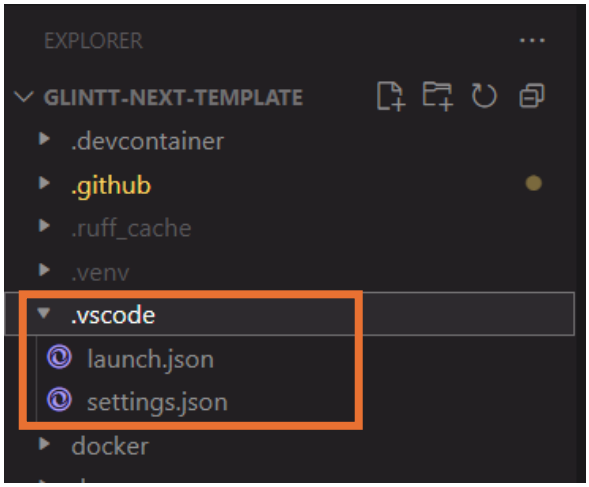
The --install-hooks isn't necessary the first time the code is run, but afterwards if you want to install anything again, maybe due to your wish to change the version or whatever, it will be needed, so it is good practice just to have it. Another good practice is to run the tests locally before the commit, because say you write your whole commit message, and if the test fails you will have to do it all again, so checking locally first can prevent this.

uv run pre-commit run -a

This will run the tests, the -a flag is optional as after each run it will cache the results, so if it has gone over a file and it doesn't detect a change then it will ignore/skip the file.

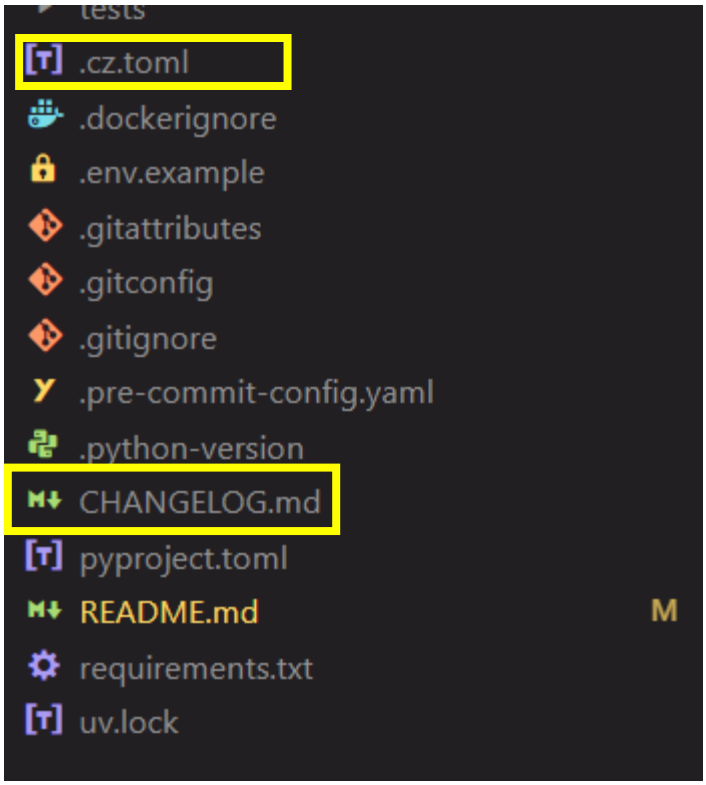
Note that the exclude part at the top of the file defines files we want to exclude from this check, in this case we only have the README, as sometimes errors in the README can be annoying to fix and obscure the incentive to use these tools.

Vscode Config



.vscode folder a very simple package that defines two files, one is the launch.json which configures how the debugger in vscode should activate when being used, what library to use and what configurations. This can also be done manually, but this way automates that process for general use cases, for specific ones configuring manually might be better. The settings.json configures the IDE, which extensions to use, which environment OS distro to use, how to use intellisense and so on.

Commitizen



Commitizen files are coloured yellow, it is a very useful tool to organize commit structure, logging and semantic version control. It follows the conventional commit standards, which is internationally recognized. **.cz.toml** is the main powerhouse that defines how the interface will look when you are creating the commit, and CHANGELOG.md keeps track of the versions of the project, a shorter form of Release Notes. It follows standard semantic versioning protocol.

More information on how commitizen works and its various tools can be found in its documentation on Github here. We will talk about the two commands that will be used most.

Starting off the commit workflow, we want to prepare the commit message. This will be done in a few stages, firstly the type of change the commit is introducing will be asked (feature, documentation etc) then the scope of the commit, this corresponds to the files, folders in question. Then a short description, less than 12 words to explain the change. A longer description can be provided in the next section. Then a question will be asked if the change is a breaking change, this is relevant for semantic versioning because if for example the change is a MAJOR change, then the first number 0.x.x gets bumped, i.e. to 1.x.x but if the change is a breaking one, then the version doesn't get bumped. Finally, a footer, which is where you would right down which Issue does it address if any, and does it fix the issue, does it close it…. All of this done when you type the following

uv run cz commit

A cool video showing how it works can be found here.

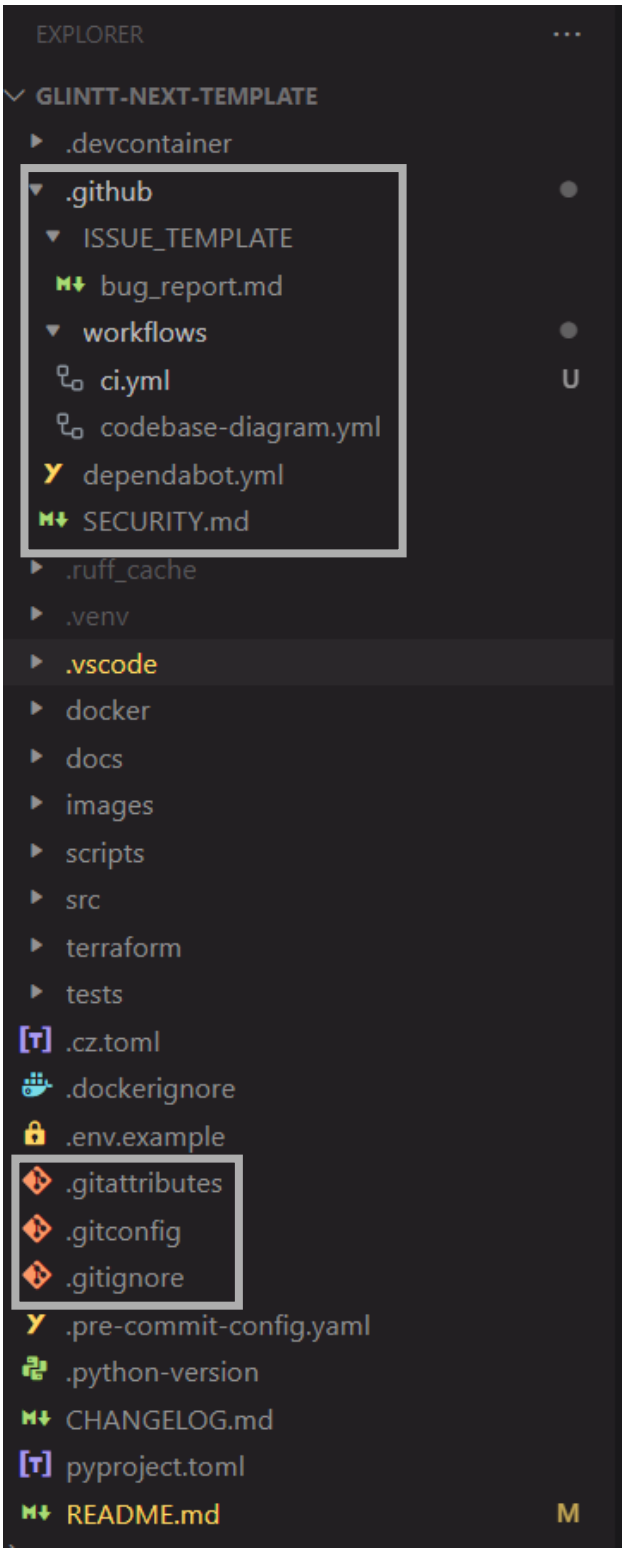
Next, we move onto the semantic versioning, reminder that all these settings, the messages, sentences displayed in the commit interface, the potential options etc are all customizable, but in the template package as of now it is only the default version. Just like the commit interface is the default version, the semantic versioning is also default, meaning it follows the MAJOR.MINOR.PATCH rule. It then based on your conventional commit formatting and the type of change you specified it will bump the respective number or add a sub-indexing e.g. 1.1.1a.

This is done with the following command, and once again a cool video on how it works is found here. (**NOTE, you have to do this before you push, because it reads the staged commit, so it needs to have that there**)

uv run cz bump

git files and .github folder

Another big section, this will focus mainly on the actual git native tools and features that they provide.



Firstly we have the **Security.md** file, this defines the template for the Security tab, in Github which shows stuff related to supported versions / tags of the project and then things like problems reported with specific versions. (Figure 1 below)

Next we have the **dependabot.yml**, which defines rules for specific Dependabot implementation, which tracks and lets you know if dependencies in your project are not up-to-date / deprecated. (Figure 2 below)

Github Workflows are Actions which get triggered when a certain git event occurs (push to main, pull request,…) and can be used t define CI/CD flows. E.g. checking the docker image builds successfully, checking the code is properly linted etc, the latest docker image is pushed to the Azure Container Registry…..this area is very useful to define these workflows, and are defined in the **workflows/ folder.** Workflows in action can be seen in Figure 3 below.

Template files, define templates for Issues and Pull Requests, which get automatically used when these are opened. You can see examples in Figure 4 below of how it looks. The markdown version is what gets implemented here.

The .**gitattributes, .gitconfig** define ease of use, git aliases and simplifications when using git in the repo. **.gitignore** has the files that you don't want in the remote repository, like environment files and sensitive information, also very big files for testing for example, rendered cache….and so on

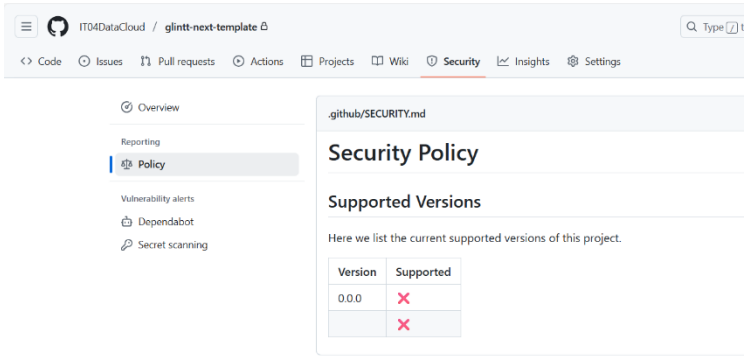


Figure 1: Security Policy for Reporting Errors / Version enabled by Security.md

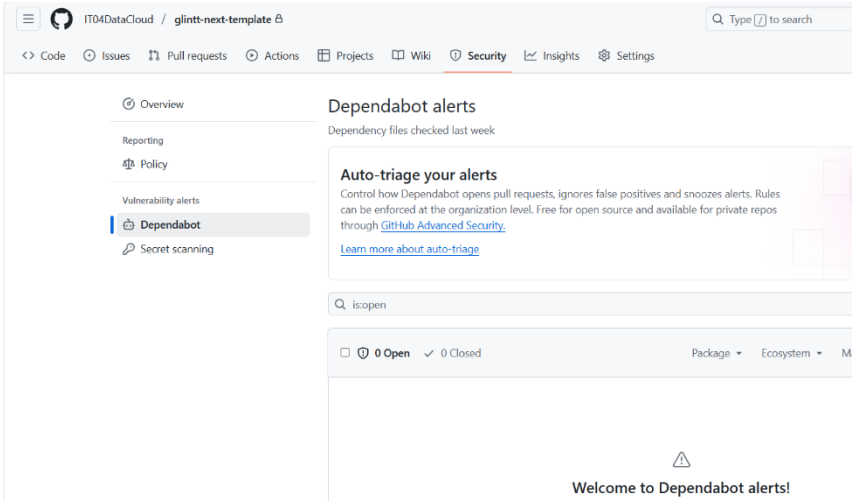


Figure 2: Dependabot Tab in Github, showing you the dependency tree and where problems with deprecated / unsafe dependencies will be shown

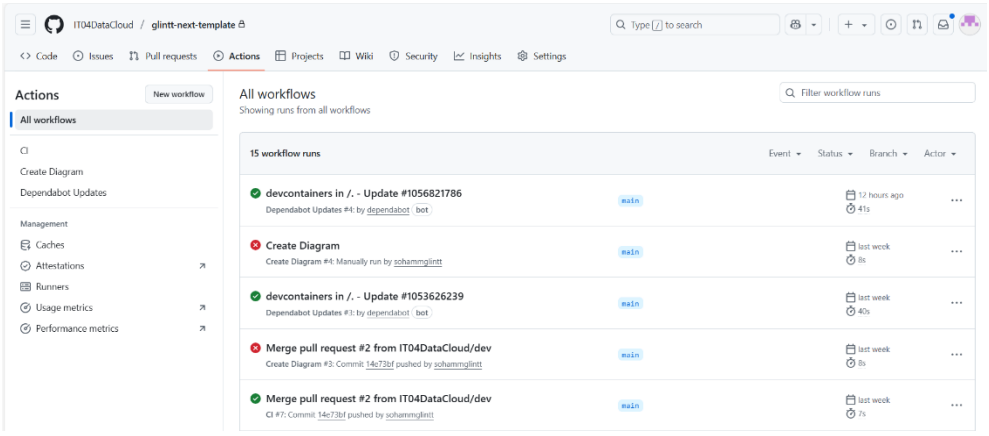


Figure 3: Github Actions tab, where the Workflows are executed and demonstrated if succeeded with a green tick and a red cross if not

Github Specific Things

Now let's leave the project and head over to this project on GitHub. Some comments on what we see and how to understand the policies in place, some mentioned in the previous section, are executed and found.

Branching Strategy for the Project

This section describes the branching strategy to be used in our GitHub repositories. A visual representation can be found in Figure 5 below.

- **Main branches**
  - **Production Branch (main/master):** The primary branch should always represent a production-ready state. No direct commits are allowed on this branch; instead, changes must be merged through **pull requests** (PRs) after code review, testing and QA approval.
- **Development branch**
  - Serves as the **staging area** for new features and bug fixes. **All completed work from feature branches should be merged into this branch first**. This allows for comprehensive integration testing and verification before changes are promoted to the production branch. The integration branch also acts as a conduit for Jira updates, ensuring that commits linked with specific Jira issue keys automatically update the corresponding tasks.
- **Feature branches**
  - These are automatically created by JIRA when an issue is moved to In Progress.
  - Always based on the latest development branch
  - Each branch should focus on a **single issue** or **task**
  - After development is complete: PR is opened to the **development branch** and when the **PR is merged,** the JIRA issue is moved to **Done.**

These branches need to be protected (done with Rulesets, where to find this is shown in Figure 4 below). Before merging, they must pass the following:

- Code review
- Unit tests
- Functional tests
- Documentation updates

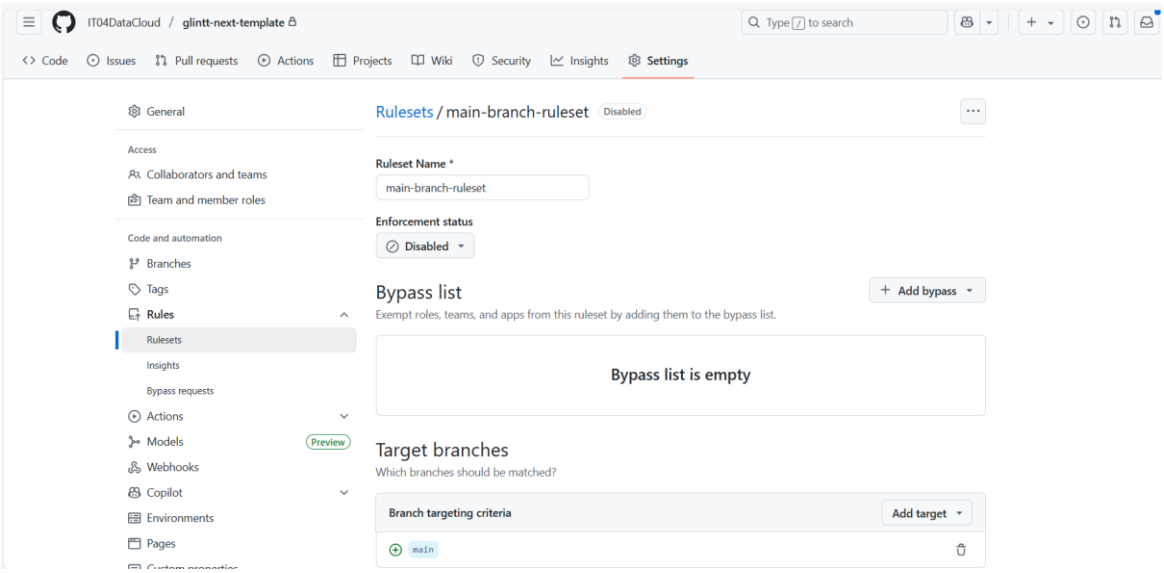


Figure 4: Find rulesets by going to settings, Rules, Ruleset and clicking on the ruleset that appears, scroll down to see configuration settings

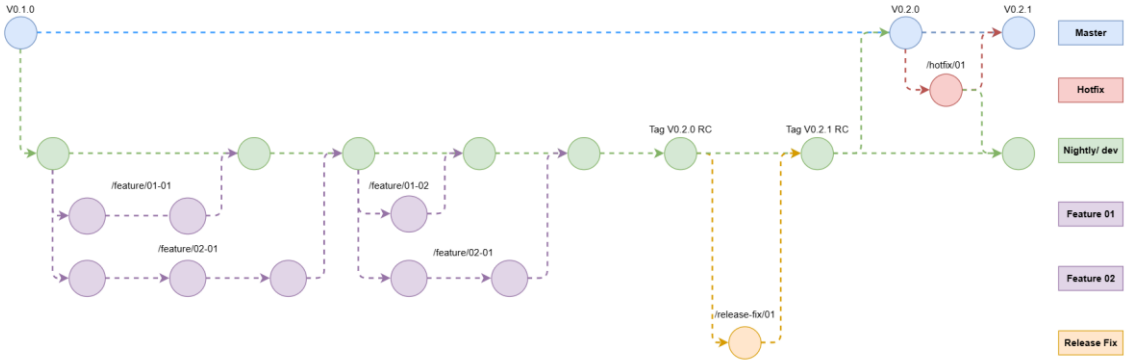


Figure 5: Visual representation of Branch Naming Scheme to follow

Integration of Github with Jira

Some JIRA concepts to be familiar with, **Backlog**. It's where all tasks, bugs, improvements, and ideas related to the project should be registered. It must be regularly updated and prioritized. Each item in the backlog should include:

- A clear and descriptive title
- A concise and actionable description
- Links to related GitHub issues or pull requests
- A defined issue type (e.g., Story, Bug, Task)
- An associated Epic (if applicable)

Next we have **Sprints**, which help structure the execution of prioritized work over fixed periods (commonly 1–2 weeks). Best practices for sprint management:

- Only include tasks that are ready for development (clear, estimated, and prioritized)
- Conduct a Sprint Planning session at the beginning of each sprint
- Hold a Sprint Review and Retrospective at the end to improve continuously
- Aim for a realistic commitment, not maximum capacity loading

Finally, we have **Epics**, which are used to group related tasks under a larger feature, goal, or initiative. They help:

- Structure work at a high level (e.g., "Set up CI/CD pipeline", "MVP Frontend", "Azure Key Vault integration")
- Track progress on strategic deliverables
- Improve clarity and reporting over long-term efforts

Each task/story should be linked to a relevant Epic whenever applicable for better tracking. **Integrating Jira with GitHub** enhances collaboration by automating workflows, improving visibility, and ensuring that development work is directly tied to project management tasks. The integration allows for seamless tracking of issues, pull requests, and releases, ensuring better coordination between development and project management teams.

We will use JIRA automation rules integrated with GitHub to streamline the development workflow and ensure traceability between code and tasks. The goal is to reduce manual work and keep the project board always up to date.

Below are the automation rules that should be considered:

- Create branch when "In Progress"
  - **Trigger**: Issue moved to In Progress in JIRA
  - **Action**: Automatically creates a new branch in GitHub.
  - The branch is created from the development branch.
  - **Naming convention:** *<type>/<jira-key>-<short-description>*
  - This branch is then used by the developer to implement the feature or fix

- On Pull-Request merge move to "Done"
  - **Trigger**: Pull Request is merged into the target branch (development)
  - **Action**: Move the related JIRA issue to the "Done" column
  - Ensures only merged and deployed code is considered completed

An example showing how this looks in practice is shown below in Figure 6.

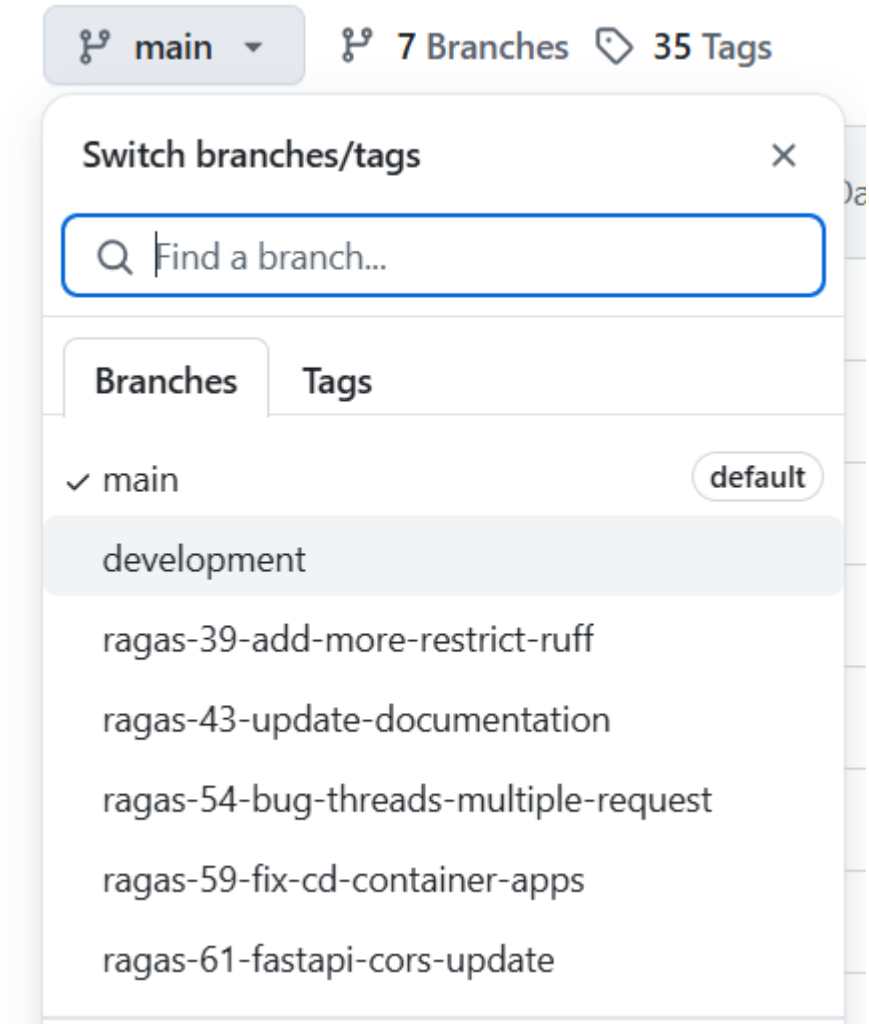


Figure 6: Jira Github Integration Branch Naming Rules, with naming convention.

Terraform

Not included here, as theses files are too dependent on the project structure, but having an overview understanding of how this Infrastructure-as-Code language works will be useful for you when starting a project. It will automate deployment and resource destruction.

Workflow, How to send local changes to remote repo

Now this is the part where you will refer to when you just want to get started, and what to see ok, how do I go from local changes to those changes being integrated in the remote repository. Firstly, ensure you are working on the correct branch. Always create a branch for new feature, and branch from dev. As mentioned, the main branch should never be touched, only once the project is deployed and fully functional on a production scale. Now once you have made your local changes, make sure the correct functionality is visible in the devcontainer and its not only working in your local environment (this can be skipped if the change isn't major like a README edit). Start by opening a terminal in your repository and typing.

Note in the following, using "uv run" is only necessary if the required dependencies aren't installed locally, it is best practice to add uv run but it isn't necessary.

git status

Check if everything makes sense, then proceed to add these files to a staged step

git add -A // or git add <specific_files>

Once again do a status check to see if everything is as it should, and all desired files have been staged

git status

Now that we have verified that the staged files are correct, we can run a local pre-commit check to see if everything is correct.

uv run pre-commit run -a

If everything is correct and passes, we can proceed to the next step. If this is not the case, check the error and fix it in the code. Make sure to re-add the file with git add, and then re-run the pre-commit check until everything is green. Note that pre-commit is configured to exclude changes to README.md for example, so if it is a readme change then can move on to the next step with commitizen.

uv run cz commit

Go through the cz commit building and once the whole message is generated, and the commit is made it may take some time which is due to the pre-commit which are being run. Note that pre-commit is run whenever git commit is called, and cz commit does in fact call that but under the hood, if we call it directly, we will circumvent this feature which isn't good. That is why we should define aliases in our shells so that this doesn't happen. After this, run the semantic versioning feature of bumping, to update the version tag, as it needs a staged commit to do this. Because in the .cz.toml we have defined we want to update the changelog automatically we don't need the (--changelog) tag

uv run cz bump

All good, now the commit is ready, and the version is also ready. We just need to push these changes to the remote repository.

git push (--set-upstream origin/<branch_name> local_branch_name)

Add the set-upstream to link the branches so in the future only git push needed. Now we have successfully pushed to the remote repository, now any remaining further checks are done on the Github level. We can now go to github and see the push.



Now what other situations to be aware of outside of our local terminal scope? Refer to the **Github Specific Things** Chapter above for information on what can happen, and where they will be displayed in Github.

Resources

uv documentation: uv

ruff documentation: Ruff

commitizen documentation: Commitizen